

Введение

Разработка видео игр является одним из самых сложных процессов в творческой индустрии. Создание игры включает проработку геймплея, разработку графики, анимаций, интерфейса, написание сюжета и множество других процессов. Однако самым важным процессом является написание программного кода. Именно благодаря коду задуманные геймдизайнерами механики и придуманный писателями сюжет воплощаются в игровую реальность а нарисованные художниками персонажи оживают. От качества программного зависят сроки разработки, производительность, возможность поддержки и перспективы дальнейшего развития игры. Плохо написанный и организованный код приводит к увеличению количества ошибок и багов, усложняет добавление новых механик и значительно замедляет разработку игры. Тогда как хорошо написанный и организованный код позволяет быстрее внедрять новые механики, снижает количество багов и ошибок и делает игру более устойчивой к изменениям. Важнейшую роль в написании и организации кода играет архитектура. Архитектура определяет структуру проекта, какие инструменты и системы будут использоваться, как они будут взаимодействовать и принципы написания кода. От выбранной архитектуры зависят удобство поддержки, масштабируемость и производительность игры. Грамотно спроектированная архитектура позволяет ускорить разработку, сократить затраты на поддержку и обеспечить возможности развития игры без необходимости значительной переработки уже существующего кода.

Несмотря на стремительное развитие трёхмерной графики, 2д игры продолжают оставаться востребованным направлением игровой индустрии. Современные 2д представлены практически во всех жанрах: платформерах, головоломках, стратегиях, RPG и многих других жанрах. Не менее важным является значительное присутствие двухмерных проектов на рынке мобильных игр. Многие успешные двухмерные игры последних лет демонстрируют, что качественная реализация художественной стилистики, игровых механик, сюжетной составляющей важнее сложной технической реализации или графики

Особую актуальность тема проектирования архитектуры приобретает при разработке 2д игр. Несмотря на всего два измерения и сравнительно простую графику, современные 2д игры могут содержать сложные игровые механики, искусственный интеллект, сложный интерфейс, сетевое взаимодействие и множество других функций. Без продуманной архитектуры разработка и поддержка игры становится значительно сложнее а поддержка и расширение требуют значительно больших временных затрат.

Не менее важным фактором при разработке игры является выбор игрового движка. На сегодняшний день существует большое количество движков для создания 2д игр, среди которых наиболее популярными являются Unity, Unreal Engine, Godot и GameMaker. Каждый из этих движков обладает своими преимуществами и недостатками. От используемого движка зависят язык программирования, доступные инструменты

разработки, способы организации проекта и возможности реализации игровых механик.

Таким образом исследование архитектурных подходов, принципов и паттернов проектирования является важной и актуальной задачей, потому что именно архитектура определяет качество технической части игры и во многом влияет на успешность игры.

Цель работы - исследование архитектурных подходов, принципов и паттернов проектирования, применяемых при разработке 2д игр, а так же изучение популярных движков для разработки 2д игр.

Для достижения задуманных целей необходимо решить следующие задачи

- изучить теоретические основы архитектуры игровых приложений
- рассмотреть требования и факторы для выбора архитектуры
- исследовать основные принципы проектирования используемые при разработке игр
- проанализировать современные архитектурные подходы используемые при разработке игр
- изучить паттерны проектирования, применяемые в разработке 2д игр
- исследовать преимущества и недостатки популярных движков для разработки 2д игр
- разработать практический пример архитектуры игры на выбранном движке
- выполнить анализ преимуществ выбранного архитектурного решения

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АРХИТЕКТУРЫ ДЛЯ ИГР

1.1 Значение архитектуры для разработки игр

Архитектура игры представляет собой совокупность решений, принципов и правил, определяющих организацию и структуру кода, взаимодействие её компонентов и принципы реализации функциональности. Архитектура определяет, каким образом будут организованы различные части проекта, как они будут взаимодействовать между собой и каким правилам необходимо следовать при разработке новых систем и компонентов.

Основной задачей архитектуры является организация кода таким образом, чтобы различные части игры могли эффективно функционировать и взаимодействовать между собой. В процессе разработки необходимо обеспечить совместную и эффективную работу множества подсистем, среди которых система управления персонажем, искусственный интеллект, пользовательский интерфейс, аудиосистема и другие компоненты [3]. Без заранее спроектированной и продуманной архитектуры каждая новая система может разрабатываться по-разному. Это приводит к появлению большого количества зависимостей между различными частями проекта. В результате

код становится сложным для понимания, тестирования и дальнейшей поддержки. Грамотно спроектированная архитектура обеспечивает повышение читаемости программного кода, упрощение сопровождения проекта, возможность повторного использования компонентов, снижение связанности между системами, упрощение тестирования и повышение масштабируемости проекта.

Архитектура оказывает влияние не только на процесс разработки, но и на весь жизненный цикл игры. После выпуска игры разработчикам приходится исправлять ошибки, выпускать обновления и добавлять новый контент. Если архитектура была изначально не была рассчитана на расширение системы, подобные изменения могут потребовать значительной переработки уже существующих систем и компонентов [4]. Особенно важную роль архитектура играет при командной разработке. Следование общим архитектурным принципам и правилам гарантирует совместимость разных компонентов и систем и упрощает интеграцию разных компонентов и систем в один проект. Это позволяет разработчикам создавать разные системы и компоненты параллельно, а так же значительно уменьшить время на адаптацию и обучения. Кроме того, архитектура оказывает влияние на производительность игры. Неправильная организация взаимодействия между объектами может привести к избыточному потреблению памяти и вычислительных ресурсов, что приведёт к ухудшению производительности игры и соответственно повлияет на комфорт игрока.

Таким образом, архитектура является фундаментом любого игрового проекта и оказывает непосредственное влияние на качество, производительность и удобство поддержки игры. Поэтому выбор архитектуры является одним из наиболее важных этапов разработки игры. Ошибки, допущенные на данном этапе, могут привести к увеличению сроков разработки, усложнению поддержки, к ухудшению производительности и появлению большого количества багов и ошибок [4]. Одним из основных требований к архитектуре является масштабируемость. Программная система должна позволять добавлять новые игровые механики и функции без необходимости полной переработки уже существующего кода. Не менее важным требованием является модульность. Архитектура должна обеспечивать разделение проекта на независимые компоненты и системы, каждая из которых отвечает за выполнение своей определённой задачи. Это снизит зависимость компонентов и систем друг от друга и позволит использовать их повторно. Важным требованием так же является поддерживаемость игры. В процессе разработки игры регулярно возникает необходимость исправления ошибок, изменения существующих механик и внедрения новых. Архитектура должна обеспечивать возможность выполнения подобных задач с минимальными затратами времени. К числу дополнительных требований относятся низкая связанность компонентов, читабельность кода, удобство тестирования, надёжность работы и производительность.

При выборе архитектуры необходимо учитывать размер проекта, количество разработчиков, сроки разработки, предполагаемое количество игровых механик, требования к производительности и используемый игровой движок. Небольшие проекты зачастую не требуют сложной архитектуры. В то же время крупные проекты

нуждаются в более продуманных решениях, обеспечивающих удобство дальнейшего развития игры. Также необходимо учитывать жанр будущей игры. Например, платформеры и головоломки обычно содержат ограниченное количество игровых объектов и могут использовать относительно простые архитектурные подходы. Стратегии, ролевые и многопользовательские игры требуют более сложных подходов из-за большого количества игровых сущностей и взаимодействий между ними [9]. Кроме того, необходимо учитывать возможности используемого игрового движка. От движка зависит язык программирования и доступные архитектурные подходы.

Следовательно, выбор архитектуры должен осуществляться с учётом особенностей конкретного проекта и требований, предъявляемых к технической части игры.

1.2 Принципы проектирования архитектуры для игр

Принципы проектирования представляют собой набор правил и рекомендаций, позволяющих создавать качественные, гибкие и удобные код. Использование данных принципов помогает уменьшить сложность проекта, повысить читаемость кода и упростить дальнейшее развитие проекта. В игровой разработке принципы проектирования особенно важны, поскольку современные игры содержат большое количество взаимодействующих между собой систем и компонентов.

Одним из наиболее известных наборов принципов проектирования являются принципы SOLID. Их основная задача заключается в создании гибкой и расширяемой архитектуры, устойчивой к изменениям.[3]

Принцип единственной ответственности SRP (Single Responsibility Principle) предполагает, что каждый класс должен отвечать только за одну задачу. Если класс отвечает сразу за несколько различных обязанностей, его становится сложнее поддерживать и изменять [8]. Например, система управления персонажем не должна одновременно отвечать за сохранение игрового прогресса или работу пользовательского интерфейса.

Принцип открытости-закрытости OCP (Open Closed Principle) требует, чтобы компоненты были открыты для расширения, но закрыты для изменения [7]. Это позволяет добавлять новый функционал не изменяя уже существующий код, что уменьшает вероятность появления ошибок.

Принцип подстановки Лисков LSP (Liskov Substitution Principle) предполагает что объекты производных классов могли использоваться вместо объектов базового класса без нарушения логики работы программы [7]. Соблюдение данного принципа делает код более гибким и модульным

Принцип разделения интерфейсов ISP (Interface Segregation Principle) предполагает что классы не должны зависеть от ненужного им функционала интерфейса. Вместо этого предполагается использовать специализированные интерфейсы [7]. Благодаря этому классы реализуют только действительно необходимую функциональность и не зависят от методов, которые им не нужны.

Принцип инверсии зависимостей DIP (Dependency Inversion Principle) предполагает зависимость от абстракций, а не от конкретных реализаций [7]. Это позволяет уменьшить связанность между компонентами и облегчает замену отдельных частей системы без необходимости изменения остального проекта.

Помимо SOLID в игровой разработке активно применяется принцип SoC (Separation of Concerns). Основная идея данного принципа заключается в разделении программы на независимые части, каждая из которых отвечает за собственную область ответственности [7]. Например, система пользовательского интерфейса должна заниматься только отображением информации, система управления персонажем - обработкой действий игрока, а система сохранений - работой с данными. Такой подход способствует повышению модульности проекта и упрощает поддержку кода.

Принцип DRY (Don't Repeat Yourself) направлен на устранение дублирования кода. Основная идея данного принципа заключается в том, что одна и та же логика не должна реализовываться в нескольких местах программы одновременно. Если одинаковый код используется в разных частях проекта, его рекомендуется выносить в отдельные методы, классы или компоненты для повторного использования [8]. Это позволяет уменьшить объём кода, повысить его читаемость и упростить дальнейшую поддержку проекта. Кроме того, при необходимости внесения изменений разработчику потребуется изменить код только в одном месте, что значительно уменьшает вероятность появления ошибок. В разработке игр данный принцип часто применяется при создании систем и компонентов управления персонажем, пользовательского интерфейса и различных механик, которые могут использоваться сразу несколькими объектами.

Принцип KISS (Keep It Simple, Stupid) предполагает создание максимально простых решений без неоправданного усложнения архитектуры. Данный принцип предполагает что необходимо выбирать наиболее простое решение задачи, которое соответствует требованиям архитектуры проекта. Нередко разработчики создают слишком сложные системы и компоненты в попытке предусмотреть будущие возможности расширения проекта. Однако подобный подход зачастую приводит к увеличению количества кода, усложнению архитектуры и снижению читаемости программы. Простые решения легче понимать, тестировать и поддерживать, а также значительно проще обучать новых разработчиков работе с проектом. Особенно важным при разработке небольших и средних по размеру игр, где чрезмерное усложнение архитектуры может принести больше проблем, чем пользы [8].

Принцип YAGNI (You Aren't Gonna Need It) рекомендует не реализовывать функции, которые не требуются на текущем этапе разработки. Достаточно часто разработчики пытаются заранее предусмотреть большое количество возможностей, которые могут понадобиться в будущем. В результате в проекте появляются неиспользуемые классы, системы и компоненты, усложняющие архитектуру и увеличивающие объём кода. Следование принципу YAGNI позволяет сосредоточиться на разработке действительно необходимого функционала и избежать преждевременного усложнения системы. В

игровой разработке данный принцип помогает сократить сроки разработки, уменьшить количество потенциальных ошибок и сохранить архитектуру проекта более понятной и контролируемой [8] .

Совместное использование принципов SOLID, SoC, DRY, KISS и YAGNI позволяет создавать программные системы, обладающие высокой гибкостью, масштабируемостью и удобством сопровождения.

1.3 Архитектурные подходы при разработке 2д игр

При разработке современных 2д игр используется несколько архитектурных подходов, каждый из которых обладает собственными преимуществами и недостатками. Выбор конкретного подхода зависит от размера и сложности проекта, требований к производительности, особенностей геймплея и используемого игрового движка.

Одним из наиболее распространённых подходов является объектно ориентированная архитектура. В рамках данного подхода игровые объекты представляются в виде классов, содержащих данные и методы обработки поведения. Такой подход хорошо подходит для небольших проектов благодаря своей простоте и понятности. Основными преимуществами объектно ориентированной архитектуры являются использование инкапсуляции, наследования и полиморфизма [3]. Однако по мере увеличения размера проекта количество связей между классами начинает расти, что может привести к усложнению архитектуры и затруднить дальнейшую поддержку системы.

Для решения данной проблемы широкое распространение получила компонентная архитектура. В рамках данного подхода игровой объект представляет собой набор независимых компонентов, каждый из которых отвечает за определённую задачу [11]. Например, персонаж может состоять из компонентов которые обрабатывают графику, физику объекта, поведение игрока, анимации и интерфейс. Такой подход позволяет повторно использовать компоненты в различных объектах и значительно упрощает расширение функционала проекта.

Ещё одним распространённым подходом является событийно ориентированная архитектура. В данной модели взаимодействие между различными системами и компонентами осуществляется посредством передачи событий. Вместо прямого обращения систем и компонентов к друг другу создаётся событие, на которое могут подписываться заинтересованные компоненты и системы. Например, после уничтожения противника может быть отправлено событие, которое одновременно обработают система начисления очков, пользовательский интерфейс и система достижений. Такой подход уменьшает связанность между системами и компонентами и делает архитектуру более гибкой.

Так же одним из распространённых архитектурных подходов является ECS. Данный подход предполагает разделение данных и логики обработки на отдельные сущности, компоненты и системы. Entity представляет игровой объект, Component хранит данные объекта, а System содержит логику обработки этих данных [6]. Благодаря подобному разделению обеспечивается высокая производительность при работе с большим

количеством игровых объектов. Архитектура ECS активно используется в современных игровых проектах и особенно эффективна в проектах, содержащих большое количество однотипных сущностей.

Для проектов средней и высокой сложности также применяется архитектура на основе шаблона MVC. В данной модели данные, пользовательский интерфейс и логика поведения объекта разделяются на отдельные компоненты [9]. Благодаря этому уменьшается связанность между различными частями системы и упрощается тестирование кода. Однако при разработке игр классический MVC используется относительно редко и чаще применяется его модификация.

При разработке современных игр зачастую используют комбинированный подход, объединяющий преимущества сразу нескольких архитектурных подходов. Наиболее распространённой является комбинация объектно-ориентированной, компонентно-ориентированной и событийно-ориентированной архитектуры. Для крупных проектов дополнительно могут использоваться ECS, MVC и различные архитектурные шаблоны для организации кода.

Таким образом, каждый архитектурный подход обладает собственными преимуществами и недостатками. Выбор конкретного решения должен осуществляться с учётом особенностей проекта, требований к производительности и необходимости дальнейшего расширения и поддержки.

1.4 Паттерны для разработки 2д игр

Паттерны проектирования представляют собой готовые решения распространённых проблем, возникающих при разработке игр. Их использование позволяет упростить архитектуру проекта, повысить читаемость кода и избежать появления багов и ошибок.

Одним из наиболее распространённых паттернов является Singleton. Данный паттерн предполагает существование только одного экземпляра определённого класса и предоставляет глобальный доступ к нему [6]. В разработке игр Singleton часто используется для создания и предоставления доступа к различным менеджерам, например GameManager, AudioManager или UiManager.

Паттерн Observer предназначен для организации взаимодействия между объектами через события. Основная идея заключается в том, что один объект может уведомить другой объект об изменении своего состояния через событие и без прямого взаимодействия между объектами [1]. Это позволяет уменьшить связанность между системами и компонентами, а так же упростить расширение проекта.

Паттерн State используется для определения поведения объекта в зависимости от его текущего состояния. Каждое состояние представляется отдельным классом, содержащим собственную логику поведения [5]. Благодаря этому удаётся избежать большого количества условных операторов и сделать код более понятным и надёжным.

На использование паттерна State часто строится паттерн State Machine. Его основная задача заключается в обработке и управлении переходами между различными

состояниями объекта. State Machine определяет текущее состояние объекта и условия перехода между состояниями. Данный паттерн широко используется при разработке персонажей, противников и различных объектов.

По мере увеличения количества состояний обычная State Machine может становиться сложной для поддержки. Для решения данной проблемы используется Hierarchical State Machine. Данный паттерн позволяет объединять связанные состояния в иерархические группы. Например, внутри общего состояния Ground могут находиться состояния движения, атаки и ожидания предназначенные для того момента когда персонаж находится на земле. Такой подход уменьшает количество переходов между состояниями и упрощает поддержку и расширение сложной игровой логики.

Паттерн Factory используется для создания объектов. Процесс создания объектов переносится в отдельный класс и позволяет производным классам определять какой именно объект создавать [7]. Использование этого паттерна упрощает создание игровых объектов и уменьшает связанность компонентов системы.

Для повышения производительности игры часто используется паттерн Object Pool. Вместо постоянного создания и уничтожения объектов они используют уже созданные объекты повторно через специальный пул [1]. Особенно полезен данный подход для массового создания временных игровых объектов, например: пуль, визуальных эффектов и других подобных объектов.

Ещё одним распространённым паттерном является Command. Данный паттерн позволяет передавать действия и условия для их выполнения в виде отдельных объектов [1]. Такой подход позволяет передавать, обрабатывать и вызывать в нужном порядке действия без прямого взаимодействия компонентов и систем с друг другом. Этот подход способствует уменьшению связанности между компонентами и системами.

Важную роль выполняет паттерн Dependency Injection Container. Основная задача данного паттерна заключается в управлении зависимостями между различными частями системы [12]. Вместо получения зависимостей через прямое взаимодействие объект получает их извне через специальный контейнер. Благодаря этому уменьшается связанность между компонентами, упрощается тестирование и повышается гибкость архитектуры.

Для организации процесса запуска игры часто применяется паттерн Entry Point. Данный паттерн предполагает наличие специальной точки входа, которая отвечает за первоначальную настройку игры. В процессе запуска создаются основные сервисы, регистрируются зависимости и выполняется инициализация игровых систем. Использование Entry Point позволяет централизовать процесс запуска проекта и сделать его более контролируемым.

Для взаимодействия между компонентами и системами часто используется паттерн Event Bus. Его основная задача заключается в организации обмена событиями между компонентами и системами игры. Вместо прямого обращения одной системы к другой

создаётся общий контейнер через который передаются сообщения между различными частями игры [5]. Такой подход значительно уменьшает количество зависимостей между системами и облегчает дальнейшее расширение архитектуры.

Несмотря на многочисленные преимущества паттернов проектирования, их использование должно быть оправданным. Избыточное применение шаблонов способно привести к усложнению архитектуры и снижению читабельности кода. Поэтому рекомендуется использовать только те паттерны, которые действительно необходимы для решения задач конкретного проекта.

На сегодняшний день существует большое количество игровых движков, поддерживающих разработку 2д игр. Каждый из них обладает собственными преимуществами и недостатками, набором встроенных инструментов и архитектурными подходами. Среди наиболее актуальных движков для создания 2д игр можно выделить Unity, Unreal Engine, Godot и GameMaker.

Unity является универсальным игровым движком, который поддерживает разработку как двухмерных, так и трёхмерных игр. Движок предоставляет широкий набор встроенных инструментов для работы с графикой, анимацией, физикой и пользовательским интерфейсом. Одним из его преимуществ является большая количество дополнительных инструментов, библиотек и ассетов, развитое комьюнити и низкий порог входа. К недостаткам можно отнести сравнительно высокие требования к ресурсам при разработке больших или сложных игр, отсутствие некоторых инструментов и необходимость платить за лицензию [2].

Unreal Engine представляет собой мощный игровой движок, ориентированный преимущественно на создание сложных игр. Он предоставляет широкий набор инструментов направленный на создание сложных и графически впечатляющих игр. Так же из преимуществ можно отметить инструменты визуального программирования. Однако движок имеет высокие требования к производительности, а для двухмерных игр возможности движка зачастую оказываются избыточными, так же его освоение требует большего количества времени по сравнению с другими движками.

Godot является бесплатным игровым движком с открытым исходным кодом. Он активно развивается и предоставляет удобные инструменты для создания двухмерных игр. Основными преимуществами Godot являются удобство и простота использования, низкие требования к производительности, низкий порог входа и отсутствия необходимости платить за лицензию. Среди недостатков можно выделить отсутствие ряда инструментов, меньшее количество туториалов, гайдов, дополнительных инструментов, библиотек и ассетов по сравнению с другими движками.

GameMaker ориентирован исключительно на разработку двухмерных игр и отличается простотой освоения и использования. Движок содержит встроенные инструменты направленные на создание простых 2д игр. Однако для разработки более сложных игр его возможности недостаточно, особенно по сравнению с другими движками.

Несмотря на различия между рассмотренными решениями, все современные игровые движки предоставляют средства для работы с графикой, физикой, анимацией и интерфейсом. Выбор конкретного движка зависит от требований проекта, опыта разработчиков, целевых платформ и особенностей разрабатываемой игры.

ГЛАВА 2. ПОСТАНОВКА ЗАДАЧ И ПРОДУМЫВАНИЕ АРХИТЕКТУРЫ ИГРЫ

2.1 Постановка задачи и выбор средств разработки

Для демонстрации на практике рассмотренных ранее архитектурных подходов, принципов и паттернов была разработана 2д игра на игровом движке Unity. Игра представляет из себя небольшую 2д стратегию с элементами строительства и защиты базы. Игровой процесс построен вокруг нескольких взаимосвязанных механик: сбора монет, найма жителей, строительства зданий и защиты базы от противников. Главные цели игрока заключаются в развитии базы, заказе строительства зданий, найме жителей и защите территории от атак противников. Основные механики игрока: перемещение, бег, подбор и выброс монет, заказ зданий и атаки. Геймплей начинается с появления игрока на уровне. На уровне размещены здания, жители, спавнер врагов, монеты и различные игровые объекты. Монеты используются как главный ресурс, за них можно нанимать жителей и заказывать постройку зданий. Здания отвечают за важные функции в игре. Одним из основных зданий является лагерь. Это здание отвечает за появление бездомных жителей. Вторым важным зданием является дворец. Данное здание выполняет роль основного объекта поселения и одновременно используется для обработки заказов на строительство. Через него осуществляется распределение задач между свободными рабочими жителями. Последним из основных зданий в игре являются стены. Стены используются для защиты поселения от атак противников. Здания строятся рабочими жителями по заказу игрока. Жители делятся на два типа: бездомных и рабочих жителей. После появления бездомные жители патрулируют территорию лагеря и ищут расположенные рядом монеты. При нахождении монеты бездомный житель подбирает её и превращается в рабочего жителя. Рабочий житель отвечает за постройку зданий. Помимо мирных жителей в игре присутствуют противники. После завершению игрового дня на карте появляются враги, которые начинают преследовать игрока. Если на их пути находится стена, они останавливаются и атакуют её. При встрече с рабочими жителями враги атакуют их. Если запас здоровья рабочего жителя заканчивается, он снова становится бездомным жителем. Исходя из описанных механик главным требованием при разработке игры было обеспечить и облегчить разработку сложно устроенных компонентов и систем. Так же одним из важных требований при разработке игры было обеспечить возможность дальнейшего расширения игры без значительной переработки и переписывания уже существующих компонентов, систем и кода. Несмотря на достаточно небольшой размер проекта, игра содержит несколько сложно взаимодействующих друг с другом систем, что делает её хорошим примером для демонстрации архитектурных подходов, паттернов и принципов. Исходя из

поставленных задач и требований для разработки был выбран игровой движок Unity. Он хорошо адаптирован и предоставляет встроенные инструменты для создания 2д игр, содержит удобную систему компонентов и поддерживает язык с#. Unity обладает большим количеством удобных встроенных инструментов для работы с анимацией, физикой, пользовательским интерфейсом. Создание игры велась на языке с#. Этот язык был выбран потому что он является основным языком разработки игр на Unity. Так же он имеет поддержку объектно ориентированного программирования и имеет большое количество библиотек и встроенных средств для написания и организацию архитектуры и кода игры. Visual Studio была использована как среда разработки. Этот редактор предоставляет полную интеграцию с Unity, удобную навигацию по коду игры, подсветку ошибок и синтаксиса и полезные инструменты отладки. Это позволяет ускорить процесс разработки игры и упростить поиск багов и ошибок. Игра разрабатывалась и тестировалась на компьютере который использует операционную систему Windows 10. Эта операционная система была выбрана так как имеет полную интеграцию с движком Unity и множеством программ которые могут значительно ускорить разработку игры. Такой набор инструментов позволил разработать задуманные архитектурные подходы и игровые механики без нужды создания или адаптации дополнительных инструментов, систем и программ.

Вывод по главе

В начале разработки игры были выяснены ключевые механики игры и требования к архитектуре проекта. Исходя из этого были выбраны подходящий движок, среда разработки, операционная система и программы для разработки игр. Использование движка Unity предоставило широкий набор инструментов, библиотек и компонентов для реализации задуманных механик игры и архитектурных решений. После выбора средств разработки была создана структура папок проекта, которая упрощает менеджмент и навигацию по проекту.

2.2 Выбранные архитектурные подходы и решения

Архитектура играет важную роль во время разработки игры. Поэтому при разработке проекта ей было уделено особое внимание. Несмотря на небольшой размер игры, в ней присутствуют множество разных игровых сущностей такие как: игрок, жители, противники и здания. Каждая из этих сущностей имеет собственную логику и поведение. Разработка такой игры без правильно выбранной архитектуры может привести к созданию сильно зависимого и сложного для чтения кода, и соответственно к появлению множества багов, ошибок и усложнению расширения и поддержки игры. Поэтому была выбрана архитектура на основе объектно ориентированного подхода, иерархии наследования, машины состояний и контейнера зависимостей. Для структурирования и упрощения навигации по игре используется структура папок. В качестве точки старта для игры используется SceneHandler. SceneHandler хранит основную информацию о сцене и создаёт глобальные классы сцены. Например именно в нём создаётся класс GeneralLinksManager. Для создания и хранения глобальных сервисов и информации для всей игры используется класс GeneralLinksManager.

Основу для каждой сущности задаёт иерархия наследования. Она используется для избежания дублирования кода и упрощения создания новых объектов и сущностей а так же для гарантирования совместимости их с разными системами и компонентами. Иерархия наследования задаёт структуру, параметры, и принципы для реализации каждой сущности в игре. Для обработки и организации логики поведения игровых сущностей используются состояния. Каждое состояние представляет из себя отдельный класс который отвечает только за выполнение конкретной логики поведения. Для обработки и управления переключениями состояний используется машина состояний. Для организации связей между объектами используется контейнер зависимостей. В проекте имеются два вида контейнеров: глобальный и локальный. Глобальный контейнер используется для хранения объектов, которые могут понадобится во всех частях игры. Локальные контейнеры применяются для хранения объектов, необходимых конкретной игровой сущности. Для уведомления о разных событиях в игре компонентам, системам и сущностям без создания прямой связи используется шина событий. Так же важную роль имеют интерфейсы. Они позволяют взаимодействовать разным классам без прямой связи с друг другом через внутренние методы интерфейсов.

Вывод по главе

В результате использования объектно ориентированного подхода, иерархии наследования, машины состояний, контейнера зависимостей удалось получить достаточно простую архитектуру, соответствующую размеру и требованиям игры. После продумывания архитектуры проекта стало возможным перейти к её реализации.

ГЛАВА 3. РЕАЛИЗАЦИЯ ОСНОВНЫХ АРХИТЕКТУРНЫХ РЕШЕНИЙ

3.1 Организация проекта и структура исходного кода

Одной из важных вещей при разработки игр является проектирование структуры папок проекта. Даже для небольшой и не особо сложной игры наличие понятной и удобной структуры папок позволяет сильно упростить навигацию по проекту и загружать необходимые файлы исходя из их логического расположения. Главным местом для хранения ресурсов игры используется папка Resources. В ней находятся подпапки которые хранят различные игровые объекты, материалы, скрипты и анимации игры. Так же имеено через эту папку Unity загружает запрашиваемые через скрипт объекты во время игры. Для хранения заготовленных игровых объектов используется подпапка Prefabs. Для хранения графики, анимации и дополнительных компонентов игровых сущностей и объектов была создана подпапка ObjectsComponents. Папка Animations отвечает за хранения анимаций и связанных с этим компонентов. В свою очередь папка Sprites хранит 2д графику проекта. Большая часть скриптов находятся в подпапке Scripts. Подпапка Interfaces содержит интерфейсы, используемые различными объектами проекта. В подпапке EventBus находятся скрипты связанные с реализацией системы обмена событиями. Скрипты связанные с реализацией контейнера

зависимостей находятся в подпапке DiContainer. Базовые родительские и главные скрипты расположены в папке BaseScripts. В папки Objects находятся скрипты игровых существ и логика их поведения. Внутри эта папка делится на подпапки для игрока, жителей, противников и зданий. Каждая из этих существ имеет свои состояния. Состояния каждой из этих существ находятся внутри подпапок State, которые в свою очередь находятся внутри папок этих существ. Благодаря этому упрощается поиск необходимых скриптов или состояний связанных с определённой сущностью. Сцены игры располагаются в папке Scenes.

Вывод по главе

На этом этапе разработки игры была создана структура папок которая логически разделяется папки и подпапки по функции и принадлежности. Подобное разделение позволяет легко находить файлы и компоненты зная их функцию и принадлежность и даёт возможность удобно реализовывать задуманные архитектурных решения.

3.2 Реализация иерархии наследования

Иерархия наследования играет важную роль в разработке игры. Несмотря на сравнительно небольшой размер игры, в ней присутствует несколько различных типов игровых существ: игрок, противники, жители и здания. Каждая из этих существ обладает собственной логикой поведения, параметрами и набором состояний. При разработке таких существ без использования наследования пришлось бы писать один и тот же код для каждой сущности отдельно, что существенно усложнило бы поддержку и расширение игры. По этой причине в качестве основы архитектуры была выбрана иерархия наследования, которая позволяет вынести общие методы, параметры и принципы в родительские классы, а особенности реализации оставить конкретным реализациям игровых существ. В разработанной игре иерархия наследования используется практически для всех игровых существ. Она определяет единые правила создания существ, хранения данных, обработки состояний и взаимодействия с другими системами игры. Благодаря этому каждая новая игровая сущность автоматически получает уже реализованный функционал и должна реализовать только собственную игровую логику. Такой подход значительно уменьшает количество повторяющегося кода и обеспечивает одинаковую структуру реализации всех объектов проекта. Основой всей иерархии игровых существ является абстрактный класс BaseObject. Он представляет собой общий родительский класс для всех игровых существ независимо от их назначения. Данный класс задаёт минимальный набор методов, который обязан реализовать каждый наследник. К ним относятся методы инициализации и старта сущности, загрузки и сохранения данных. Кроме этого, BaseObject задаёт виртуальный параметр ObjectData предоставляющее доступ к типу данных соответствующей сущности. Для хранения информации обо всех игровых объектах используется отдельная ветвь наследования, основой которой является класс BaseObjectData. В отличие от BaseObject этот класс не отвечает за выполнение игровой логики, а предназначен исключительно для хранения данных игровых существ. В нём располагаются минимальный набор параметров и свойств.

Например там располагаются идентификатор сущности и параметр `ObjectSaveData` который предоставляет доступ к данным для сохранения каждой сущности. Такое разделение логики и данных соответствует принципу разделения ответственности и делает архитектуру проекта более понятной. Следующим уровнем данной иерархии является класс `BaseObjectSaveData`. Он представляет собой общий родительский класс для всех данных для сохранения. Каждый наследник содержит идентификатор и идентификатор типа сущности, что позволяет в дальнейшем расширить систему сохранений без изменения архитектуры игровых сущностей. После определения базовой структуры объектов начинается разделение игровых сущностей по их функциональному назначению. Следующим классом иерархии является `BaseDynamicObject`. Он наследуется от `BaseObject` и предназначен для всех объектов, которые обладают собственным поведением во время игрового процесса. Помимо методов базового класса он добавляет абстрактные методы `ObjectUpdate` и `ObjectFixedUpdate`. Использование отдельного промежуточного класса позволяет разделить статические и динамические объекты проекта. Если некоторому игровому объекту не требуется постоянное обновление логики, наследование от `BaseDynamicObject` становится необязательным. Благодаря этому архитектура остаётся достаточно гибкой и позволяет создавать различные типы игровых объектов без изменения уже существующей структуры. Следующим уровнем наследования является класс `BaseEntity`, который становится общим родительским классом для всех конкретных игровых сущностей проекта. Именно от него в дальнейшем наследуются игрок, жители, противники и здания. Помимо функциональности `BaseDynamicObject` данный класс содержит общий экземпляр системы сохранения и определяет абстрактные методы загрузки и сохранения конкретной сущности. Параллельно с `BaseEntity` располагается соответствующий класс хранения данных `BaseEntityData`, который является родительским классом для всех данных игровых сущностей. Именно этот класс объединяет несколько наиболее важных компонентов архитектуры проекта. Внутри него создаётся экземпляр `EntityStateMachine`, который отвечает за обработку состояний конкретной сущности. Благодаря этому каждый объект автоматически получает собственную машину состояний сразу после создания класса данных. Так же в `EntityDataFacade` располагается собственный экземпляр `DiContainer`. В отличие от глобального контейнера сцены данный контейнер используется только конкретной игровой сущностью. Во время её инициализации в контейнер помещаются ссылки на необходимые компоненты Unity, такие как `Animator`, `Rigidbody2D`, `AnimationEventsReceiver` и другие объекты, которые впоследствии могут потребоваться различным состояниям. Все необходимые зависимости передаются через контейнер, что значительно уменьшает связанность различных частей проекта. Последним базовым классом данной ветви наследования является `BaseEntitySaveData`. Он наследуется от `BaseObjectSaveData` и становится общей основой для всех структур данных сохранения конкретных игровых сущностей. В дальнейшем именно от него наследуются классы `PlayerSaveData`, `EnemySaveData`, `VillagerSaveData` и `BaseBuildingSaveData`. Использование отдельного родительского класса обеспечивает единую структуру хранения данных и позволяет при необходимости расширять систему сохранений сразу

для всех игровых сущностей без изменения классов игрока, противников, жителей или зданий. Одной из наиболее важных игровых сущностей является класс `Player`. Он наследуется от класса `BaseEntity`, благодаря чему автоматически получает поддержку машины состояний, системы сохранения и общего жизненного цикла объекта. Сам класс `Player` практически не хранит игровые параметры. Его основной задачей является получение ссылок на компоненты `Unity`, создание объекта данных, регистрация необходимых зависимостей в локальном контейнере и запуск машины состояний. Во время инициализации в контейнер зависимостей помещаются ссылки на компоненты `Animator`, `AnimationEventsReceiver` и `Rigidbody2D`, после чего производится настройка основных параметров игрока и запуск машины состояний. Такое разделение обязанностей позволяет сделать класс `Player` относительно компактным и сосредоточить большую часть игровой информации в отдельном классе данных. Для хранения всех параметров игрока используется класс `PlayerData`, который наследуется от `BaseEntityData`. Именно в данном классе располагается практически вся информация, необходимая для работы игрока. Здесь хранятся скорость, урон, направление движения, ссылки на компоненты интерфейса, собранные монеты и множество других параметров. Кроме этого, `PlayerData` содержит экземпляр класса `PlayerInputAction`, параметры анимации, ссылки на объекты сцены и события, используемые различными состояниями игрока. Также именно здесь определяется состояние, с которого начинает работу машина состояний игрока. Благодаря подобному подходу все данные, используемые различными состояниями, располагаются в одном месте, что значительно упрощает их использование и дальнейшее сопровождение. Для хранения информации, необходимой системе сохранения, используется класс `PlayerSaveData`, наследующийся от `BaseEntitySaveData`. В нём располагается идентификатор типа сущности, а также данные, необходимые для сохранения состояния игрока. Аналогичным образом реализована и архитектура противников. Основным классом игровой сущности является `Enemy`, который также наследуется от `BaseEntity`. Как и `Player`, данный класс отвечает преимущественно за создание объекта данных, получение необходимых компонентов `Unity` и запуск машины состояний. Благодаря использованию одного и того же родительского класса игрок и противники имеют практически одинаковую структуру реализации, несмотря на полностью различающуюся игровую логику. Все параметры противника располагаются в классе `EnemyData`, наследующемся от `BaseEntityData`. В нём содержатся характеристики здоровья, скорости движения, наносимого урона, направления движения, параметры обнаружения препятствий и объектов, ссылки на точки проверки столкновений и необходимые события. Помимо этого, `EnemyData` определяет набор параметров анимации, ссылки на игровой объект и стартовое состояние машины состояний. Для хранения информации о сохранении противников используется класс `EnemySaveData`, который наследуется от `EntitySaveData`. Он определяет идентификатор типа сущности и обеспечивает совместимость противника с общей системой сохранения проекта. Благодаря использованию общей цепочки наследования сохранение различных игровых сущностей реализуется одинаковым способом независимо от их назначения. Наиболее сложной по внутренней структуре

игровой сущностью являются жители. Основным классом данной системы является Villager, который также наследуется от BaseEntity. Как и остальные игровые сущности, он отвечает только за создание объекта данных, регистрацию зависимостей и запуск машины состояний. Для хранения информации о жителе используется класс VillagerData, наследующийся от BaseEntityData. В данном классе сосредоточено наибольшее количество параметров среди всех игровых сущностей проекта. Здесь располагаются ссылки на различные варианты игровых объектов жителя, компоненты Unity, данные о текущем заказе, ссылки на спавнер, параметры обнаружения окружающей среды, характеристики движения, стоимости найма, событий строительства и получения приказов. Кроме этого, класс содержит реактивные свойства и события, позволяющие различным состояниям отслеживать изменение компонентов без создания жёстких зависимостей между ними. Благодаря такому подходу вся информация о текущем состоянии жителя находится в одном объекте и может использоваться различными состояниями без дублирования данных. Для системы сохранения используется класс VillagerSaveData, который наследуется от BaseEntitySaveData. Его назначение аналогично классам PlayerSaveData и EnemySaveData: определение типа сохраняемого объекта и хранение информации, необходимой системе сохранения. Отдельную ветвь иерархии наследования составляют здания. В отличие от остальных игровых сущностей все здания используют промежуточный родительский класс BaseBuilding, который наследуется от BaseEntity и одновременно реализует интерфейсы IDamagable и IBuildable. Такое решение связано с тем, что независимо от назначения каждое здание обладает одинаковым набором базовых возможностей. Любое здание может получать урон, участвовать в процессе строительства, содержать информацию о стоимости улучшения и сообщать другим системам о возможности начала строительства. Вынесение этого функционала в отдельный родительский класс позволило избежать повторной реализации одинаковых методов во всех конкретных зданиях проекта. Все данные зданий располагаются в классе BaseBuildingData, который наследуется от BaseEntityData. Помимо функциональности, полученной от родительского класса, BaseBuildingData содержит параметры, общие для всех зданий. К ним относятся уровень здания, запас здоровья, информация о доступных улучшениях, ссылки на игровые объекты, радиус проверки окружающей среды, пользовательский интерфейс, а так же на события начала строительства и получения урона. Благодаря этому каждое конкретное здание хранит только собственные уникальные параметры, тогда как вся общая информация определяется родительским классом. Последним элементом данной цепочки является BaseBuildingSaveData, наследующийся от BaseEntitySaveData. От него наследуются классы данных сохранения всех конкретных зданий проекта, например PalaceBuildingSaveData, WallBuildingSaveData и CampBuildingSaveData. Использование общего родительского класса обеспечивает одинаковую структуру сохранения данных всех зданий.

Вывод по главе

Была реализованна иерархии наследования которая позволила сформировать единую архитектуру проекта, в которой каждая игровая сущность реализуется по одинаковым правилам. Общий функционал был вынесен в родительские классы, а конкретная логика располагается только в конкретных наследниках. Благодаря этому удалось значительно сократить количество повторяющегося кода, упростить взаимодействие различных систем игры и обеспечить возможность дальнейшего расширения игры. Это позволило сделать код проекта более структурированным, повысить его читаемость и существенно упростить дальнейшую поддержку и развитие игры. Создания качественной основы для игровых сущностей позволило перейти к созданию логики их поведения.

3.3 Реализация системы состояний

Одним из ключевых элементов архитектуры разработанной игры является система состояний. В процессе игрового процесса каждая игровая сущность постоянно изменяет своё поведение в зависимости от происходящих событий. Например, игрок может находиться в состоянии ожидания, передвижения, атаки или получения урона. Противник способен патрулировать территорию, преследовать и атаковать игрока. Жители изменяют своё поведение в зависимости от выполняемой профессии, полученных приказов и наличия рядом противников. Реализация подобной логики при помощи большого количества условных операторов значительно усложняет сопровождение проекта, увеличивает связанность различных частей программы и делает добавление новых возможностей очень сложным процессом. По этой причине для управления поведением игровых сущностей была выбрана система состояний. Основной задачей системы состояний является разделение сложного поведения объекта на независимые логические блоки, каждый из которых отвечает только за выполнение одной конкретной задачи. Вместо хранения всей логики внутри одного класса, каждая логика поведения была вынесена в отдельные состояния. Основой всей системы является класс `BaseStateMachine`. Именно он управляет жизненным циклом состояний, выполняет их инициализацию, обновление и переключение. Машина состояний создаётся для каждой игровой сущности отдельно и располагается внутри соответствующего класса данных, благодаря чему каждый объект обладает собственной независимой системой управления поведением. Такое решение позволяет нескольким объектам одного типа одновременно находиться в различных состояниях и работать независимо друг от друга. При создании машины состояний передаётся ссылка на главное состояние сущности. После завершения инициализации вызывается метод старта, который активирует текущее состояние. В дальнейшем при каждом обновлении игрового цикла машина состояний вызывает методы текущего состояния, полностью делегируя ему выполнение логики поведения объекта. При необходимости смены поведения машина состояний завершает работу предыдущего состояния и передаёт управление новому состоянию. Благодаря этому в каждый момент времени выполняется только одно активное состояние, отвечающее за текущее поведение игровой сущности. Следующим уровнем архитектуры является абстрактный класс `BaseState`, который представляет собой основу всех состояний проекта. Именно данный

класс определяет общий жизненный цикл любого состояния независимо от его назначения. Он содержит методы инициализации, входа в состояние, обновления и завершения работы состояния. Каждый конкретный наследник обязан реализовать данные методы в соответствии со своей игровой логикой. Благодаря этому переключение между состояниями осуществляется универсальным механизмом без необходимости учитывать особенности конкретного класса. Так же BaseState содержит ссылку на родительскую машину состояний и предоставляет возможность переключения между состояниями через общий механизм переходов. В результате сами состояния не взаимодействуют напрямую друг с другом. Для смены поведения достаточно передать машине состояний ссылку на новое состояние, после чего она самостоятельно завершает текущее состояние и активирует следующее. Подобное решение значительно уменьшает связанность различных частей проекта и делает архитектуру более гибкой. Одной из особенностей разработанной системы является наличие дополнительного уровня абстракции BaseGrandState. В отличие от обычных состояний данный класс не описывает конкретное действие игровой сущности, а объединяет состояния в одну логическую группу исходя из задуманного назначения. Такое решение было принято из-за того, что некоторые игровые объекты обладают несколькими крупными режимами поведения, внутри которых располагается собственный набор состояний. BaseGrandState наследуется от BaseState и одновременно содержит собственную локальную систему переключения состояний. Помимо стандартных методов жизненного цикла данный класс хранит базовое состояние группы и текущее активное состояние. Таким образом, великое состояние представляет собой отдельную подсистему внутри общей машины состояний. Для основной машины оно выглядит как одно обычное состояние, однако внутри него может находиться любое количество дополнительных состояний и переходов между ними. Внутри BaseGrandState состояния делятся на два типа: базовые состояния и текущее состояние. Базовое состояние это группа состояний которые обрабатывают ключевую логику великого состояния в котором находятся одновременно с текущим состоянием и не могут быть убраны или изменены. Текущее состояние это состояние которое обрабатывается какую то логику поведения игровой сущности и может быть изменено. Благодаря такому подходу значительно уменьшается сложность всей архитектуры, поскольку каждая группа состояний рассматривается как самостоятельный логический модуль. Кроме этого использования BaseGrandState даёт возможность разделить различные режимы поведения игровых объектов. Например, для жителя отдельно реализованы состояния бездомного и рабочего. Каждый из этих режимов содержит собственный набор состояний, которые никак не пересекаются между собой. Машина состояний переключается только между великими состояниями, тогда как все переходы внутри конкретного режима выполняются локальной системой данного великого состояния. Наиболее простой структурой обладает Player. Для него создаётся собственное великое состояние PlayerGroundGrandState. Оно наследуется от BaseGrandState и содержит ссылки на все основные состояния игрока. В процессе инициализации создаются базовые состояния обработки окружающей среды, управления инвентарём, управлением здоровьем и стандартное текущее состояние.

Каждое обычное состояние игрока наследуется от BaseState и отвечает только за выполнение одной конкретной задачи. Например, состояние передвижения управляет перемещением персонажа, состояние атаки выполняет обработку нанесения урона. Благодаря разделению поведения на небольшие независимые классы каждая часть логики становится значительно проще для сопровождения и дальнейшего расширения. Аналогичная архитектура используется и для противников. Главным состоянием является EnemyGrandState. При инициализации EnemyGrandState создаются базовые состояния, включающие проверку окружающей среды, управлением здоровьем и стандартное текущее состояние. Каждое из них представляет собой отдельный класс, наследующийся от BaseState и реализующий только собственную игровую логику. Например состояние преследования игрока которое отвечает за преследование игрока и атаку стен, игрока и жителей , или состояние смерти которое отвечает за смерть противника.

Вывод по главе

Так была реализованна система состояний которая представляет из себя систему управления поведением игровых сущностей, основанный на последовательном разделении логики на несколько уровней: машина состояний отвечает за обработку и переключение между состояниями игровой сущности, великие состояния объединяют состояния в группы по задуманому назначению, а состояния обрабатывают игровую логику.

3.4 Реализация контейнера зависимостей

Одним из важных элементов архитектуры разработанной игры является контейнер зависимостей. В процессе разработки проекта количество взаимосвязей между игровыми объектами, системой состояний и компонентами игрового движка постепенно увеличивается. Игрок, противники и жители используют компоненты анимации, физики, системы событий, машину состояний и другие компоненты Unity, которые необходимы различным игры. Если каждое состояние самостоятельно будет получать необходимые компоненты с помощью методов поиска, программный код станет сильно связанным, что значительно усложнит его сопровождение и дальнейшее развитие. Для решения данной проблемы в проекте был реализован собственный контейнер зависимостей, обеспечивающий централизованное хранение и предоставление необходимых объектов различным подсистемам игры. Контейнер зависимостей представляет собой специальный класс, предназначенный для регистрации объектов и последующего предоставления их другим компонентам программы. Основная задача контейнера заключается в устранении прямых зависимостей между различными элементами архитектуры. Вместо самостоятельного поиска необходимых компонентов игровые состояния получают их через единый механизм разрешения зависимостей. В разработанной игре контейнер зависимостей используется всеми игровыми сущностями и располагается в классе BaseEntityData. Каждый экземпляр игрока, противника или жителя содержит собственный объект данных, внутри которого создаётся отдельный контейнер зависимостей. Такое решение

обеспечивает независимость игровых объектов друг от друга, поскольку каждый из них использует только собственный набор зарегистрированных компонентов. Благодаря этому изменение состава компонентов одной сущности никак не влияет на работу остальных объектов, находящихся на игровой сцене. Одним из наиболее наглядных примеров использования контейнера является класс `Player`. Во время выполнения метода `ObjectInitialize` создаётся экземпляр класса `PlayerData`, после чего в его контейнер зависимостей регистрируются основные компоненты игрового объекта. В контейнер помещаются ссылки на `Animator`, `AnimationEventsReciever`, `Rigidbody2D` и другие компоненты которые могут понадобиться для работы состояний игрока. Благодаря этому все необходимые зависимости состояния получают непосредственно из контейнера. Аналогичный это реализованно и в классе `Enemy`. Во время создания противника создётся экземпляр `EnemyData`, содержащий собственный контейнер зависимостей. После этого в контейнер помещаются ссылки на компоненты `Animator`, `AnimationEventsReciever`, `Rigidbody2D` и другие объекты, необходимые состояниям противника. Затем состояния преследования игрока и смерти используют уже зарегистрированные зависимости. Наиболее эффективно преимущества контейнера зависимостей проявляются при реализации жителей. В отличие от игрока и противников, поведение жителя разделено между двумя великими состояниями: `HomelessVillageGrandState` и `WorkerVillageGrandState`, каждое из которых содержит собственный набор базовых и обычных состояний. Несмотря на большое количество отдельных классов, все они используют один экземпляр контейнера зависимостей, расположенный внутри `VillagerData`. Благодаря этому состояния бездомного и рабочего жителя получают доступ к необходимым компонентам, игровым объектам и параметрам персонажа через удобную точку доступа. Такой подход позволил полностью разделить два различных режима поведения персонажа, сохранив при этом единый способ доступа к контролю и предоставлению зависимостей. Например, состояние передвижения игрока использует зарегистрированный компонент `Rigidbody2D` для движения персонажа, состояние атаки получает компонент `Animator` для запуска соответствующей анимации, а состояния жителей используют зарегистрированные ссылки на игровые объекты и вспомогательные компоненты без обращения к классу `Villager`. Не мало важным преимуществом реализованного решения является возможность дальнейшего расширения игры. При добавлении новой игровой логики или нового состояния отсутствует необходимость сильно изменять существующий код для добавления необходимых им компонентов. Достаточно зарегистрировать новую зависимость в контейнере во время инициализации объекта, после чего она автоматически становится доступной всем состояниям, которым требуется данный компонент. Такой подход соответствует принципу открытости и закрытости объектно ориентированного программирования и позволяет развивать проект без существенного изменения уже существующего кода.

Вывод по главе

Был создан контейнер зависимостей который является одним из ключевых элементов архитектуры созданной игры. Его использование позволило централизовать

предоставление доступа различных компонентов и существенно уменьшить связанность различных частей игры. Благодаря применению контейнера зависимостей система состояний, игровые сущности и компоненты Unity взаимодействуют посредством единого механизма регистрации и получения компонентов, что делает архитектуру проекта более модульной, удобной для сопровождения и готовой к дальнейшему расширению.

3.5 Реализация системы строительства

Система строительства является одной из наиболее важных систем игры. Именно она задаёт глубину стратегического геймплея и позволяет игрокам почувствовать развитие базы. Изначально все здания находятся в не построенном состоянии. Для начала строительства игроку нужно подойти к нужному зданию. После этого игрок должен отдать необходимую сумму монет за строительство. Игрок не занимается строительством самостоятельно, а только заказывает строительство зданий. После оформления заказа здание которое надо постоить переходит в состояние `BuildBuildingState`. Данное состояние отвечает за обработку процесса строительства самого здания и хранит необходимую информацию для строительства. Далее информация о заказе передаётся в `PalaceBuilding`. Как Центральное здание базы, `PalaceBuilding` занимается обработкой множества процессов включая обработки заказов на строительство и распределение задач между свободными рабочими жителями. Для обработки заказов используется состояние `OrderTreatmentPalaceState`. Его задача заключается в поиске свободного рабочего жителя, который способен выполнить строительство. Если подходящий житель найден, ему передаётся информация об обрабатываемом заказе. После получения задания рабочий переходит в состояние `WorkVillagerState`. Именно в этом состоянии происходит выполнение строительства зданий. Сначала рабочий направляется к зданию которое надо построить. После достижения цели, рабочий начинает процесс строительства здания. В течение определённого времени выполняется процесс строительства, после чего здание переходит из непостроенного состояния в построенное и начинает выполнять свою функцию. После завершения задания житель освобождается и снова может получать новые заказы. Эта система предоставляет разделение ответственности между различными участниками процесса. Игрок создаёт заказ, `PalaceBuilding` распределяет задачи, а рабочие жители выполняют строительство. Подобный подход позволил избежать ненужного раздывания скриптов и соответственно уменьшил шанс на появления новых ошибок и багов а так же упростил дальнейшее расширение системы. Кроме того, эта система хорошо симулирует реальное развитие поселения.

Вывод по главе

Была реализованна система строительства, которая распределяет обязанности между различными сущностями и обеспечивает выполнение строительных работ.

3.6 Оценка принятых архитектурных решений

Для оценки эффективности выбранного архитектурного подхода необходимо оценить его плюсы и минусы. Основой архитектуры стал объектно ориентированный подход, машина состояний, иерархия наследования и контейнер зависимостей. Этот выбор соответствует размеру игры и позволил избежать неоправданного усложнения проекта. Одним из главных преимуществ выбранного архитектурного подхода является разделение логики поведения объектов на отдельные состояния. Благодаря использованию машины состояний логика поведения противников, жителей и игрока располагается в отдельных классах. Это уменьшает зависимость и упрощает чтение кода. Так же не мало важным плюсом оказалось использование иерархии наследования. Общие методы и параметры были вынесены в родительские классы, что позволило сократить количество повторяющегося кода. Дополнительным плюсом стало использование контейнера зависимостей. Вместо создания сложной системы зависимостей использование контейнера зависимостей позволило с лёгкостью получать необходимые зависимости. Это уменьшило зависимость между различными частями игры и упростило создание взаимодействия между ними. Отдельно следует отметить созданную структуру папок игры. Разделение игровых сущностей и объектов по папкам и выделение отдельных папок для состояний позволило упростить навигацию по проекту. Даже при увеличении количества игровых сущностей и объектов такая структура остаётся достаточно удобной для расширения. Однако вместе с описанными плюсами так же присутствуют и минусы. Одним из таких является увеличение количества состояний и родительских классов. По мере расширения игры число скриптов может значительно увеличиться. Однако благодаря созданной структуре папок и выбранным архитектурным подходам, удастся невелировать этот минус.

Вывод по главе

Исходя из приведённых плюсов можно сделать вывод что выбранные архитектурные решения соответствуют размеру и требованиям игры. В общем выбранная архитектура успешно справилась с поставленными требованиями. Она позволила реализовать все задуманные игровые механики, сохранить приемлемую читаемость кода и обеспечить возможность дальнейшей поддержки и расширения игры.

Заключительные выводы и рекомендации

В ходе выполнения курсовой работы была достигнута поставленная цель, заключающаяся в исследовании архитектурных подходов, принципов и паттернов проектирования, применяемых при разработке двухмерных игр, изучении современных игровых движков и разработке практического примера архитектуры игрового приложения на базе движка Unity. В процессе выполнения работы были последовательно решены все поставленные задачи. Были изучены теоретические основы архитектуры игровых приложений, рассмотрены требования, предъявляемые к современной игровой архитектуре, исследованы основные принципы проектирования и наиболее распространённые архитектурные подходы, применяемые при разработке игр. Кроме того, был выполнен анализ популярных игровых движков для разработки

двухмерных игр, рассмотрены их преимущества и недостатки, а также факторы, влияющие на выбор движка при создании игровых проектов. Проведённое исследование показало, что эффективность архитектуры определяется не использованием какого либо одного подхода, а грамотным сочетанием нескольких архитектурных решений. Применение объектно ориентированного программирования, принципов разделения ответственности, системы состояний, событийно ориентированного взаимодействия компонентов и внедрения зависимостей позволяет создавать программный код, который остаётся понятным, масштабируемым и удобным для дальнейшего сопровождения даже при увеличении сложности игрового проекта. Практическая часть работы подтвердила полученные теоретические результаты. В ходе разработки была реализована собственная архитектура игрового приложения, построенная на объединении нескольких современных архитектурных решений. Основой проекта стала единая иерархия наследования игровых объектов, позволившая объединить общий функционал игрока, противников, жителей и зданий в единую систему классов. Благодаря использованию общей архитектуры удалось существенно уменьшить объём повторяющегося кода и обеспечить единый подход к разработке различных игровых сущностей. Для организации поведения игровых объектов была разработана иерархическая система состояний, включающая машину состояний, великие состояния, базовые состояния и обычные состояния. Подобное разделение позволило изолировать отдельные части игровой логики, сделать систему поведения более модульной и значительно упростить добавление новых игровых механик. Использование единой архитектуры состояний обеспечило возможность повторного применения разработанных решений при реализации игрока, противников и жителей без изменения общего принципа работы системы. Важным элементом разработанной архитектуры также стал контейнер зависимостей, обеспечивающий централизованное управление используемыми компонентами. Его применение позволило отказаться от повторяющегося поиска компонентов Unity, уменьшить связанность между различными частями программы и обеспечить единый механизм передачи зависимостей системе состояний. Совместное использование контейнера зависимостей, системы состояний и иерархии наследования позволило сформировать модульную архитектуру, удобную для сопровождения и дальнейшего развития. Полученные результаты подтверждают, что выбранные архитектурные решения позволяют повысить качество программного кода, упростить сопровождение проекта и обеспечить возможность его дальнейшего масштабирования. Реализованная архитектура успешно поддерживает добавление новых игровых объектов, состояний и игровых механик без необходимости существенной переработки существующей структуры программного обеспечения, что особенно важно при разработке проектов, функциональность которых постепенно расширяется. В процессе выполнения работы возникли определённые трудности, связанные с проектированием универсальной архитектуры игровых сущностей и организацией взаимодействия между её отдельными подсистемами. Наиболее сложной задачей стало построение общей структуры, которая могла бы использоваться различными игровыми сущностями независимо от особенностей их поведения. Для решения данной задачи была

разработана собственная архитектура, объединяющая иерархию наследования, контейнер зависимостей и трёхуровневую систему состояний в единый механизм управления игровыми объектами. Полученные результаты позволяют рекомендовать использование рассмотренных архитектурных решений при разработке игровых проектов средней и высокой сложности, в которых особое значение имеют масштабируемость, повторное использование программного кода и удобство сопровождения. Таким образом, выполненная работа подтверждает, что применение современных архитектурных подходов, принципов и паттернов проектирования является одним из основных факторов создания качественного программного обеспечения в игровой индустрии. Разработанная архитектура соответствует поставленным целям исследования, обеспечивает высокий уровень модульности и расширяемости программного кода и создаёт основу для дальнейшего развития игрового проекта без необходимости существенного изменения его программной структуры.

Библиография

1. [Nystrom , 2014] Nystrom , Robert. *Game Programming Patterns*. 2014 . ISBN 978-0990582908
2. [Uzayr , 2022] Uzayr , Sufyan. *Mastering Unity: A Beginner's Guide (Mastering Computer Science)*. 2022 , ISBN 978-1032103174
3. [Vishwakarma , 2025] Vishwakarma , Niraj. *Learning Game Architecture with Unity: Architect scalable Unity projects with proven design principles and patterns*. 2025. ISBN 978-9365898415
4. [Martin , 2017] Martin , Robert. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. 2017 . ISBN 978-0134494166.
5. [Baron , 2021] Baron , David . *Game Development Patterns with Unity 2021: Explore practical game development using software design patterns and best practices in Unity and C#*. 2021 . ISBN 978-1800200814
6. [Wilnai , 2025] Wilnai , Nitzan. *High Performance Unity Game Development using data-oriented design*. 2025. ISBN 9781633435612
7. [Lin , 2023] Lin , Wilmer . *Level up your code with game programming patterns*. 2023. Accesată la 20.06.2026, disponibilă la adresa <https://unity.com/resources/level-up-your-code-with-game-programming-patterns>
8. [Lin , 2023] Lin , Wilmer. *Create a C# style guide: Write cleaner code that scales*. 2023. Accesată la 20.06.2026, disponibilă la adresa <https://unity.com/resources/create-code-c-sharp-style-guide-e-book>
9. Digital Dragons . *Alexey Merzlikin - Architecture Behind Our Most Popular Unity Games* . Accesată la 20.06.2026, disponibilă la adresa <https://www.youtube.com/watch?v=-TlQAm8IZp4>
10. Jason Weimann (GameDev). *The 6 Design Patterns game devs need?*. Accesată la 20.06.2026, disponibilă la adresa <https://www.youtube.com/watch?v=hQE8lQk9ikE>

11. Jason Weimann (GameDev). *Unity Architecture - Composition or Inheritance?*. Accesată la 20.06.2026, disponibilă la adresa
<https://www.youtube.com/watch?v=8TIkManpEu4>
12. PracticAPI . *Reference Objects In Your Unity Game Flawlessly*. Accesată la 20.06.2026, disponibilă la adresa
<https://www.youtube.com/watch?v=xYRFwFXNskc&t=521s>